02 — Technical Requirements Document (TRD)

Product: EventPilot — The Living Plan **Status:** Proposed architecture; confirm challenge restrictions before implementation.

1. Architecture principles

- Deterministic logic owns calculations, date rules and state transitions.
- Gemini provides optional suggestions and natural-language explanations only.
- Validate all external input and model output.
- Keep the API key on the server; never embed secrets in client code.
- Prefer the smallest coherent architecture that can be built, tested and deployed within the confirmed deadline.
- Do not add authentication, database or cloud services unless required.

2. Proposed stack (provisional)

Choose one stack after confirming challenge rules:

- Frontend: TypeScript web application using a framework already supported by the starter repository. If no starter exists, choose a minimal maintained framework and avoid unnecessary dependencies.
- UI: semantic HTML, accessible components, responsive CSS; use a design system consistently.
- AI: Gemini API through a server-side endpoint, using the currently documented supported model and response schema.
- Backend: a minimal server/API route for AI calls and validation.
- Persistence: browser-local storage for MVP if allowed; no login by default.
- Hosting: select a deployment target only after confirming submission requirements and build compatibility.

Google ecosystem preference is a preference, not a reason to add services without need. Confirm model availability, pricing, quotas, terms, and deployment steps in official documentation before finalizing.

3. Logical components

1. EventSetup: validates event details.
2. PlanStore: owns event plan state and accepted changes.
3. BudgetEngine: pure functions for totals, allocation and variance.
4. ScheduleEngine: pure functions for relative dates and dependency validation.
5. ReplanEngine: compares old and proposed plan; emits typed change records.
6. AIPlannerClient: sends bounded request to server endpoint.
7. AIResponseValidator: validates model output against schema and domain constraints.
8. ProvenanceService: labels source/confidence of each value.
9. UI components: BudgetPanel, TimelinePanel, VendorChecklist, IdeasPanel, ChangeReview.
10. ErrorBoundary/StatusRegion: communicates errors and recovery options.

4. Data types (illustrative)

```
type EventType = "wedding" | "birthday" | "community";
type Provenance = "user-entered" | "ai-estimate" | "confirmed-by-user";
type TaskStatus = "todo" | "in-progress" | "done" | "blocked";

interface EventPlan {
  id: string;
```

```
  eventType: EventType;
  title: string;
  eventDate: string; // ISO date, validated
  guestCount: number;
  currency: string;
  totalBudgetMinor: number;
  priorities: string[];
  budgetItems: BudgetItem[];
  tasks: EventTask[];
  ideas: Idea[];
  updatedAt: string;
}

interface BudgetItem {
  id: string;
  category: string;
  label: string;
  amountMinor: number;
  provenance: Provenance;
  note?: string;
  locked: boolean;
}

interface EventTask {
  id: string;
  title: string;
  category: string;
  status: TaskStatus;
  dueDate: string;
  dependsOn: string[];
  priority: "low" | "medium" | "high";
  locked: boolean;
  provenance: Provenance;
}

interface Idea {
  id: string;
  title: string;
  description: string;
  rationale: string;
  estimatedCostMinor?: number;
  provenance: "ai-estimate";
}

interface PlanChange {
  id: string;
  entityType: "budget-item" | "task" | "idea" | "event";
  entityId: string;
  field: string;
  before: unknown;
  after: unknown;
  reason: string;
  requiresApproval: true;
}
```

5. API contract (illustrative)

`POST /api/plan-suggestions`

- Request: validated event type, constraints, selected plan summary and user priorities.
- Response: schema-constrained suggestions only; never send secrets or unnecessary personal data.
- Errors: `400` invalid input, `429` rate limit, `502/503` provider unavailable, `504` timeout.
- Client behavior: show recoverable status, preserve current plan, allow sample/template fallback.

The server must validate both request and provider response. Treat model output as untrusted data.

6. Budget rules

- Store amounts as integer minor units (e.g., cents/paise) where currency rules permit.
- Do not use binary floating-point as the source of truth for money calculations.
- Explicitly define whether line items are included, optional, or estimates.

- `plannedTotal = sum(included budget item amounts)`.
- `remaining = totalBudget - plannedTotal`.
- `variance = plannedTotal - totalBudget`.
- A negative remaining balance means over budget; do not silently reduce items.
- Currency-specific formatting must use the selected currency; avoid assuming all currencies have two minor digits.

7. Scheduling rules

- Use ISO date-only values for event/task dates; avoid timezone shifts caused by parsing date-only strings as timestamps.
- Validate date ranges and event date changes.
- For dependencies, reject missing task references and cycles.
- Simple relative-date scheduling is sufficient for MVP; advanced optimization is deferred.
- Locked dates must be preserved and conflicts surfaced, not silently overwritten.

8. AI safety and reliability

- Server-side API key only, loaded from environment/secret manager.
- Use supported structured output/schema features documented by Google at implementation time.
- Validate schema and domain rules; bound retries and timeouts.
- Do not use AI-generated values as authoritative arithmetic.
- Render text safely; do not inject generated HTML.
- Do not claim vendor, quote, venue, weather or availability verification unless an actual trusted integration performed it.
- Rate limit and cap request sizes where supported.
- Provide a deterministic fallback and clear error state.

9. Security and privacy

- No secrets in source control, client bundles, screenshots or README examples.
- `.env.example` contains placeholder names only.
- Validate user input on both client and server.
- Avoid storing sensitive event details unless necessary.
- If local storage is used, disclose that data is stored in the browser and provide reset/clear behavior.
- Review dependency vulnerabilities and license compatibility.
- Use security headers and production-safe error messages when supported by the deployment platform.

10. Performance and efficiency

- Avoid duplicate AI calls; disable duplicate submission while a request is in flight.
- Keep calculations synchronous and local.
- Request only the context needed for a suggestion.
- Use bounded retries and timeouts.
- Cache only when safe and keyed by relevant inputs; do not let stale suggestions overwrite a newer plan.

11. Testing

- Unit: budget math, currency edge cases, date calculations, dependency validation, diff generation, locked item protection.
- Integration: API validation, malformed AI output, provider timeout, rate limiting and fallback.
- End-to-end: create plan, change guest count, review diff, reject/accept change, confirm plan state.
- Accessibility: keyboard flow, labels, focus, error associations, contrast.
- Build/lint/typecheck: all must pass before release.

12. Official verification checklist

Before implementation, check the current official Google AI docs: https://ai.google.dev/gemini-api/docs and the official challenge page: https://prompt-wars-2026.devpost.com/. Record date checked, selected model/API, restrictions, quota and cost assumptions. If not verified, keep it marked open.